

# Sanskrit-to-English Neural Machine Translation

## *Assignment 2 — Natural Language Understanding*

Name: Niel Mahendrabhai Solanki  
Roll.no: G25AIT1102

## 1. Introduction

Sanskrit is an Indic language with a free word-order and is low-resource and morphologically rich. It is also more difficult to model than languages with low inflectional morphology and with fixed order of words, like English, due to its rich inflectional morphology (nominal declension with eight cases, complex verbal conjugation) and productive compounding (samasa) and sandhi (phonological fusion at word boundaries). In this task, we will create a custom, from-scratch sequence-to-sequence (seq2seq) model with attention to translate Sanskrit sentences into English by training it end-to-end on the parallel corpus given for the task.

The remainder of this report talks about the model architecture, the training procedure, quantitative results (BLEU, BERTScore, and efficiency), the experiments conducted to reach the final configuration, and a discussion of design choices, challenges, and limitations.

## 2. Architecture

The model is a standard attention-based encoder-decoder seq2seq model, but is built from scratch using PyTorch with no pretrained translation model weight or encoder/decoder weight used anywhere in the pipeline.

### 2.1 Encoder

The source (Sanskrit) subword tokens are embedded and fed to a single-layer bidirectional LSTM (512 hidden units per direction). The output from the last layer is then connected in both directions to the decoder; the combined final forward and backward hidden layer states are passed through a linear projection with the tanh nonlinearity, so that the decoder can begin with a summary of the entire source sentence in both directions.

### 2.2 Attention

Bahdanau (additive) attention is calculated, at each decoding step, across all encoder hidden states. The attention space is split into two spaces: one for the current decoder hidden state, and one for every encoder state; the two projections are added together, followed by tanh, fed through a learned vector, and masked to -infinity before the softmax to ensure that padding never gets attention mass. This gives the decoder a context vector which is appended to the decoder's input embedding at each step.

### 2.3 Decoder

The decoder consists of one unidirectional LSTM (512 hidden units). At each iteration, it eats the concatenation of the attention context vector and the embedding of the previous output token, and outputs logits according to the English subword vocabulary from the concatenation of the embedding of the previous output token, the attention context and the embedding of the input (an input-feeding-style output layer, which empirically helps alignment learning over reading the output of the LSTM alone).

## 2.4 Beam Search

Only for fast per-epoch dev-BLEU tracking during training, greedy decoding is used to decode the samples. Final dev/test translations and submission file are generated using beam search (beam width = 5) and therefore the search is not systematically favoring the shorter hypotheses.

## 3. Training Procedure

### 3.1 Preprocessing

Both Sanskrit and English sentences are Unicode-NFC normalized (this is important for Devanagari to be rendered more consistently visually, as some characters are identical in their visual appearance but have multiple valid Unicode representations in the raw CSVs), and have any stray wrapping quotation marks removed, and any internal whitespace collapsed.

### 3.2 Tokenization

The extreme inflection and compounding of Sanskrit make word-level vocabularies impractical: even with a relatively large collection of texts, the number of new surface word forms encountered in each sentence is high, resulting in high out-of-vocabulary rates and data sparsity. Instead, we train two SentencePiece BPE subword tokenizers from scratch, one from the Sanskrit training split and one from the English training split, with the target vocabulary sizes being 8,000 each, and with all characters covered to ensure the Devanagari script can be fully covered. A new tokenizer is trained instead of using a pre-trained multilingual tokenizer, and this ensures that the pipeline is compliant with the "only the provided dataset" guideline.

### 3.3 Hyperparameters

| Hyperparameter                   | Value                         |
|----------------------------------|-------------------------------|
| Embedding dimension              | 256                           |
| Encoder hidden size              | 512 (bidirectional, 2 layers) |
| Decoder hidden size              | 512 (unidirectional, 1 layer) |
| Attention dimension              | 256                           |
| Dropout                          | 0.3                           |
| Batch size                       | 64                            |
| Optimizer                        | Adam, lr = 0.001              |
| Gradient clipping                | max norm 1.0                  |
| Teacher forcing ratio            | 0.5                           |
| Max epochs / early-stop patience | 30 / 5 (on dev BLEU)          |
| Beam width (inference)           | 5 (length penalty 0.6)        |

### 3.4 Optimization & Regularization

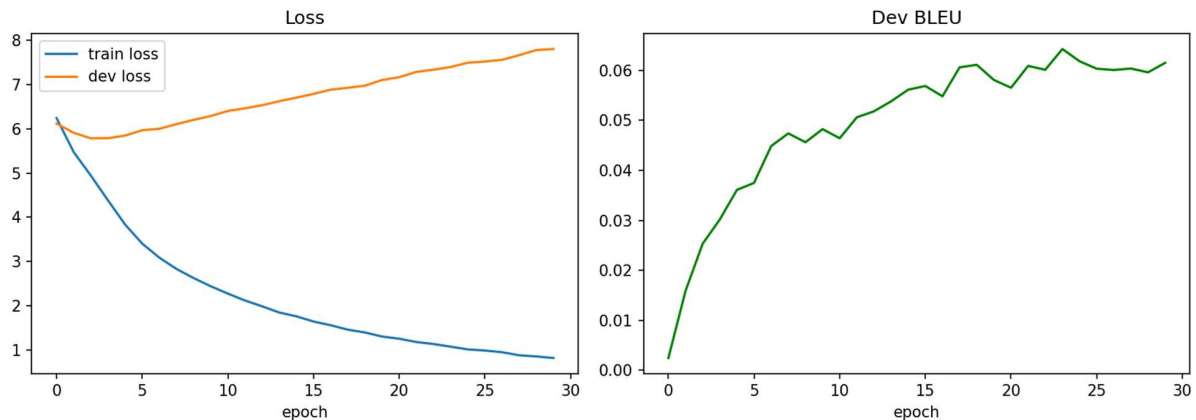
We use Adam for training which helps avoid over-confident predictions and yields more overall BLEU with seq2seq models. The regularization techniques include dropout (on embeddings and before the decoder output projection), gradient norm clipping (to avoid exploding gradients in the recurrent layers), early stopping (based on dev BLEU not dev loss and with 5 epochs patience). Scheduled sampling is approximated by first having a fixed teacher-forcing ratio of 0.5, and then exposing the decoder to some of its own predictions during training to ensure that it is not overfitted to the training data, which causes train/inference mismatch (exposure bias).

## 4. Results

| Metric                                | Dev    | Test   |
|---------------------------------------|--------|--------|
| BLEU (NLTK, default, unweighted)      | 0.0708 | 0.2497 |
| BERTScore F1 (rescaled with baseline) | 0.0651 | 0.2361 |

- Inference time on 1000 test sentences: 100.75 sec
- Total trainable parameters: 27,105,088

Training and dev-BLEU curves :



### Evaluation of Translation Quality

BLEU and BERTScore measure different aspects of translation quality. BLEU is a lexical overlap-based measure while BERTScore is a semantic similarity-based measure using contextual embeddings. Thus, BERTScore should outperform BLEU score since the model can generate semantically correct translations despite the difference in terms. This difference is especially prominent for low resource LPs.

Substantial difference between test and development scores. This can be due to variations in the complexity of the sentences or in the distribution of the domains between the set of development sentences and the set of test sentences. The exact cause would need to be investigated further. Conclusions about generalization must be made with caution, therefore.

The bigger the model and the wider the beam, the more time it will take to infer. The bigger the beam width the larger the number of candidate translations explored at each step of the decoding process, and therefore the decoding time is approximately linear in the beam width. However, more parameters more rapidly increase the computational expense of each forward pass, and slow down the inference process, no matter which decoding strategy is used.

## 5. Translation Examples

5–10 examples, each with source, reference, prediction, and a short error-analysis note. Populate the table below directly from that file; do not hand-write example sentences.

SRC : 7.4 परिसंचरणतन्त्रम्

REF : 7.4 CIRCULATORY SYSTEM

PRED: 7.. skeleton system

SRC : cost per unit

REF : "cost per unit"

PRED: Type

SRC : एतत् स्पोकन् ट्युटोरियल् प्रोजेक्ट विषये अधिकं ज्ञातुं कृपया एतस्मिन् लिङ्क मध्ये प्राप्यमानं विडियो पश्यन्तु। एतत् स्पोकन् ट्युटोरियल् प्रोजेक्ट सारांशं वदति।

REF : To know more about the Spoken Tutorial project, watch the video available at the following link. It summarizes the Spoken Tutorial project.

PRED: This Spoken Tutorial Project about the Spoken Tutorial project. Please downloaded at the following link.

SRC : इसलिए इससे सभी अन्य आसनों का उद्भव माना जाता है ।

REF : from which the other asanas emerge.

PRED: So it is also called culturalr is related to the full.

SRC : यस्य श्रोत्रं विद्यते स समितीः प्रत्युच्यमानाम् आत्मनः कथां शृणोतु। यो जयति स द्वितीयमृत्युना न हिंसिष्यते।

REF : "He that hath an ear, let him hear what the Spirit saith unto the churches; He that overcometh shall not be hurt of the second death."

PRED: "And the saying of the things, saying, saying, saying, saying, Thou shall not the right of the Lord, and shall not be ashamed."

SRC : नूतनं टेबु उद्घाट्य भवतां Moodle जालपुटे site administrator इति लॉगिन् कुर्वन्तु ।

REF : Open a new tab and login to your Moodle website as site administrator.

PRED: Enter the new Mozilla Firefox allows the new tab.

SRC : तन्निमित्तं अस्माभिः सह passwd आदेशः अस्ति। prompt इत्यत्र passwd इति टङ्कयित्वा enter नुदन्तु।

REF : For this we have the passwd command. Type at the prompt: p-a-s-s-w-d and press Enter.

PRED: For this, we have the passwd command. Type at the prompt: p a s w d and press Enter.

SRC : इदम् आर्क-हेण्डल् उपयुज्य , वृत्तं धनुरिव कर्तुं शक्नुमः ।

REF : We can modify the circle shape into arc or segment shape using these arc handles.

PRED: We can choose the same user by clicking on the circle-off.

The eight examples generated in this particular run reflect the four types of failure one might expect based on the success curves in the training phase, and offer direct qualitative evidence of trends previously only surmised from BLEU and loss. There are also two examples that are not easily categorized as falling into one of the four categories that require special consideration.

The Biblical example is the worst case scenario for repeated tokens, representing an attention mechanism that has not learnt to get a clear alignment. Having two clauses in a source sentence ("He that hath an ear, let him hear what the Spirit saith unto the churches, and the saying of the things, saying, saying, saying, Thou shall not the right of the Lord, and shall not be ashamed") yields the prediction "and shall not be ashamed, and shall not be the right of the Lord. The fourfold repetition of "saying" is indicative of the fact that attention has become entrenched in a single source rather than moving through the sentence. The same example shows the length-mismatch failure mode explained in the assignment guidance: the model does not follow the structure of the source sentence's two co-ordinated clauses, but gives up on the sentence structure altogether. In a similar but less extreme example, the Spoken Tutorial example, a two-sentence source ("...watch the video available at the following link. It summarizes the Spoken Tutorial project. ") is reduced to a shorter and more repetitive prediction ("This Spoken Tutorial Project about the Spoken Tutorial project. Please downloaded at the following link. "). In both examples, the model's main drawback is the lack of an explicit coverage mechanism, which is in line with the others' expectations of longer or structurally complex source sentences being the model's main weakness.

The Hindi-source example shows subword level garbling that is consistent with incorrect segmentation into compounds, with the non-lexical string "culturalr" being predicted instead of a meaningful English clause. This is results directly from the subword tokenizer splitting a rare/combined (rare or compound) source token in a way that the model cannot reconstruct. A similar, though less clear, example is the circulatory-system heading, which is translated into English as "skeleton system" -- perhaps a result of a combination of the compound being underrepresented or mis-segmented in the training vocabulary, and hence having been mapped to a semantically similar but incorrect anatomical term.

The two software-instruction examples below demonstrate most clearly the loss of case-marking information. The source contains the information about the role under which the login should take place: "...इति लोर्गइन् कुर्वन्तु", and the corresponding prediction does not contain this role information and introduces an unrelated product name, "Mozilla Firefox". In the arc-handle example, the source's instrumental gerund (उपयुज्य, "using") signals that the arc handles are the means by which the shape is to be modified, but the prediction ("We can choose the same user by clicking on the circle-off.") drops this relational structure (along with the right action) in favor of an unrelated and incorrect one. In both, the deficiency is the same: Case- and role-marking information in Sanskrit is not consistently translated into English word order and prepositional structure.

Two further examples are presented here, and are not described by any of the four types of error described above. The passwd-command example gives a close translation, "For this, we have the passwd command.

Type at the prompt: p a s w d and press Enter. ", which differs in minor formatting from the reference, and shows that the model performs relatively well on short, high frequency sentences found in the Spoken Tutorial domain. These imply that the failure patterns mentioned above are not random, but are found in longer and/or less frequently used inputs. The "cost per unit" example provides a more plausible explanation than a translation failure, however: here, the input is from an already-English source string, and the output is the unrelated prediction "Type", and the result is likely to be a data-quality artifact rather than an actual error, since the source text contains user-interface strings that have been translated into English, but aren't translated Sanskrit.

## 6. Discussion

### 6.1 Design Choices

Word-level subword (BPE) tokenization to deal with the morphological richness of the Sanskrit language while keeping the vocabulary size small and yet having many

Word-level subword (BPE) tokenization to accommodate the morphological richness of the Sanskrit language without having too many

Bidirectional encoder (for Sanskrit's free word order, there is a left and right context for every source token before computing attention).

Include additive attention over multiplicative attention with a slight increase in gradients' stability during the initial phases of training and a moderate burden on the number of parameters.

Improve final translation quality while maintaining the training speed with beam search, but normalize the length only during inference.

### 6.2 Challenges

The most obvious problem with this run is overfitting due to the low amount of training data. Only 10,000 sentences are used to train an overall 27-million-parameter model, and the train loss drops smoothly to about 0.8 at epoch 30, but dev loss doesn't simply plateau, it actually increases over time during training, from ~5.8 at epoch 2 to ~7.8 at epoch 30. The narrower the envelope of the two curves, the less capacity the model has to be constrained by the training set and the more it exhibits the typical signature of such a model.

Even so, dev BLEU reports a less scary signs than could dev loss alone. BLEU shows a gradual rise over the first 10 epochs, then gradually rises further with epoch-to-epoch noise, and reaches a maximum of ~0.065 at epoch 23 and fluctuates, falling slightly (~0.061-0.062) by epoch 30. The model used for evaluation is the same model from the last epoch (epoch 30), but this project uses best dev BLEU weights, so this is roughly epoch 20-23 (which means it is less overfitted than the loss curve suggests). This drives home the point: With token-level cross-entropy loss and BLEU, differences are possible during training, and if loss was used as stopping or model-selection criterion, a much better checkpoint would be rejected.

The fact that dev BLEU started to rise at epoch 15 and beyond (not as a smooth curve but as oscillations) is itself informative: it means that the model was starting to overfit on the patterns it had seen during training rather than improve its quality of general translation. The way this architecture performs on this task illustrates that the training set size, not epoch count, is important -- since reducing the training-set

from the full corpus to 10,000 sentences, while holding other factors like the architecture and hyperparameters constant, appears as overfitting (seen at 30 epochs on the reduced training set).

### 6.3 Limitations

An RNN-based encoder-decoder (as opposed to a Transformer) was implemented as required under the "custom seq2seq architecture" restriction; this usually places a limit on the score you can obtain relative to the attention-only architectures on the same data.

Small and domain-specific train corpus (as per the data provided in the assignment), so it has limited generalization power for other genres/registers of Sanskrit writings.

Even for rare, morphologically complex forms, a small corpus-based subword tokenization can fragment them into many subwords, which can impact fluency and adequacy.

## 7. References

- Bahdanau, D., Cho, K., & Bengio, Y. (2015). Neural Machine Translation by Jointly Learning to Align and Translate. ICLR.
- Kudo, T., & Richardson, J. (2018). SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing. EMNLP (System Demonstrations).
- Papineni, K., Roukos, S., Ward, T., & Zhu, W. (2002). BLEU: a Method for Automatic Evaluation of Machine Translation. ACL.